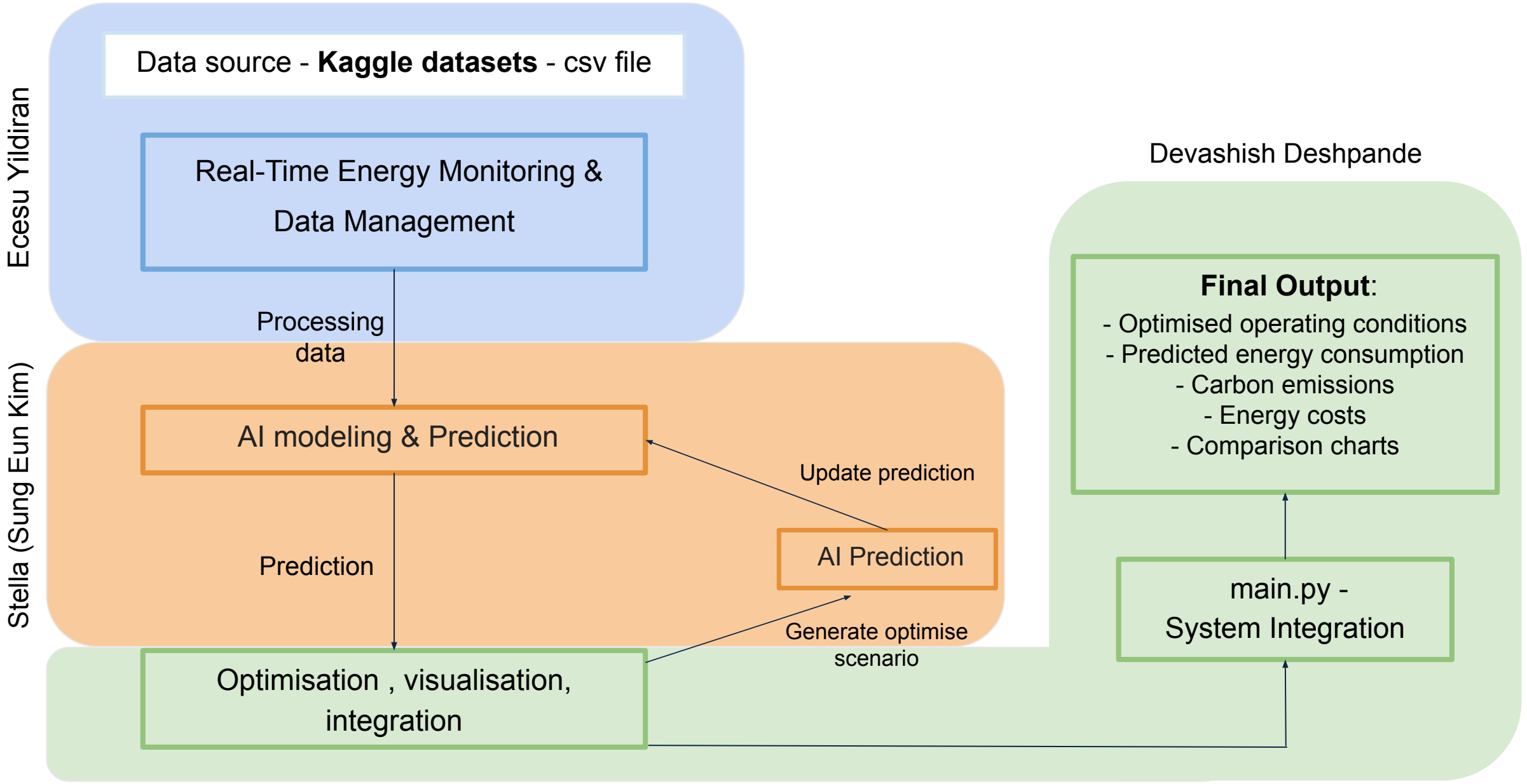
Compact Python Programming Course
Group 6

AI powered smart energy management system

Group participants:

- **Ecesu Yildiran** **[ESE 7226935]**
- **Devashish Deshpande** **[ESE 7225975]**
- **Stella (Sung Eun Kim)** **[ESE 7226046]**

AI powered smart energy management - System Architecture



Subsystem 1: **Real-Time Energy Monitoring & Data Management**

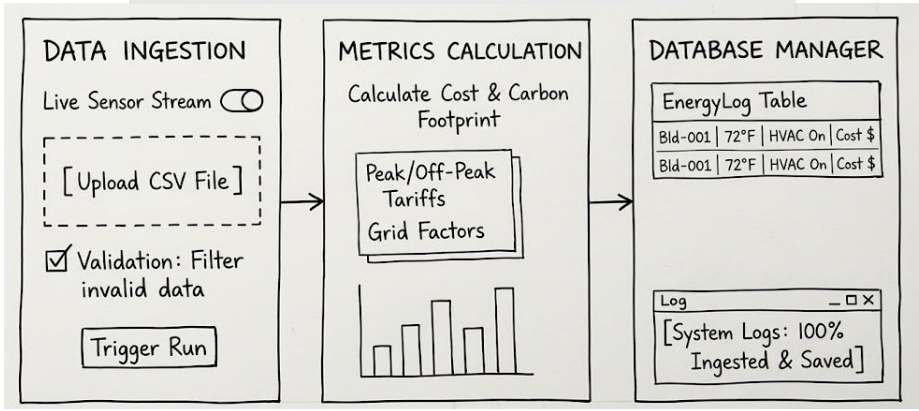
Functional Requirements

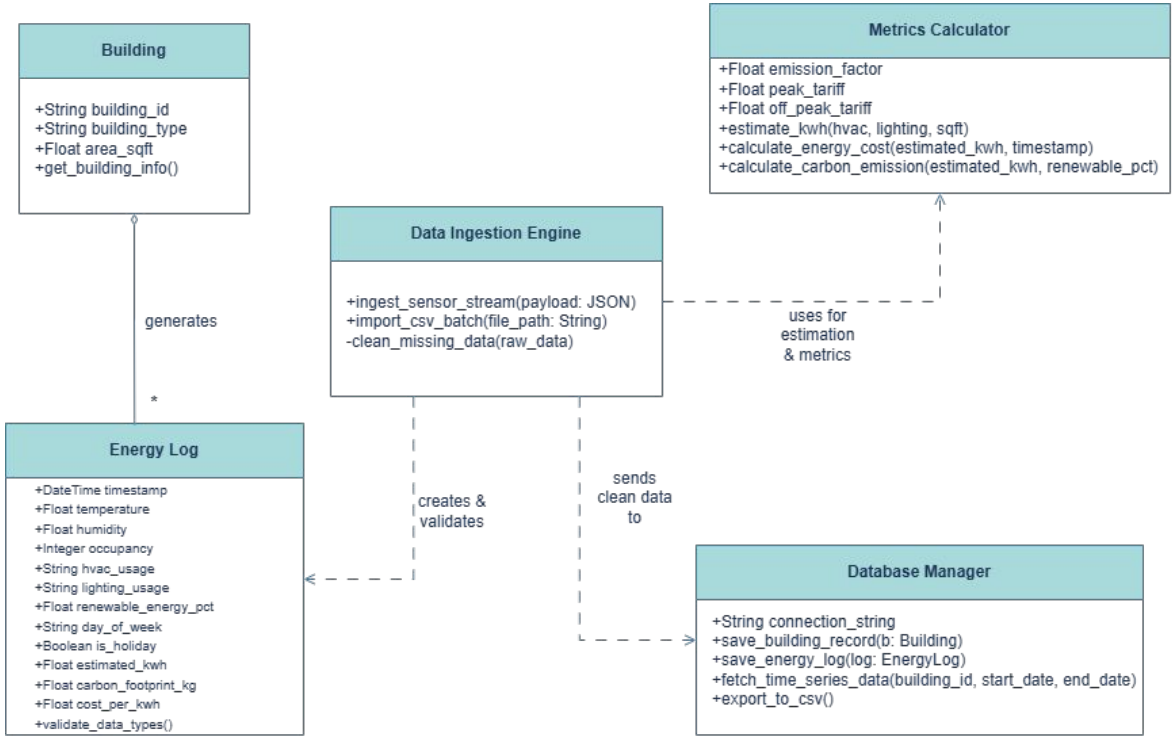
1. **Data Ingestion:** Process real-time sensor streams and batch CSV uploads.
2. **Data Storage:** Store historical usage logs (hourly, daily, monthly).
3. **Cost Calculation:** Compute energy costs using dynamic peak/off-peak tariffs.
4. **Carbon Tracking:** Calculate emissions utilizing standard grid factors.
5. **Data Validation:** Filter missing, negative, or invalid incoming data.
6. **Multi-Building Support:** Track metrics across offices, homes, and campuses.
7. **Anomaly Detection:** Automatically flag energy spikes exceeding baselines.
8. **Time-Series Queries:** Retrieve consumption data by custom date ranges.
9. **Deduplication:** Remove redundant telemetry records based on timestamps.
10. **Data Export:** Download raw and processed logs in CSV and JSON formats.

Non-Functional Requirements

1. **Performance:** <1.5s query response time for 30-day data trends.
2. **Scalability:** Support up to 100 buildings with 1-minute data sampling.
3. **Reliability:** Maintain a minimum database connection uptime of 99.5%.
4. **Data Integrity:** Enforce strict typing and database foreign key constraints.
5. **Security:** Isolate and secure credentials via environment variables (`.env`).
6. **Maintainability:** Structure logic into modular, reusable Python packages.
7. **Storage Efficiency:** Auto-compress and archive records older than 1 year.
8. **Auditability:** Log all configuration modifications with timestamps and user IDs.
9. **Interoperability:** Ensure data schemas comply with standard JSON conventions.
10. **Recoverability:** Implement automated database backup and restore scripts.

Interface Prototype





Subsystem 1: Ecesu Yildiran

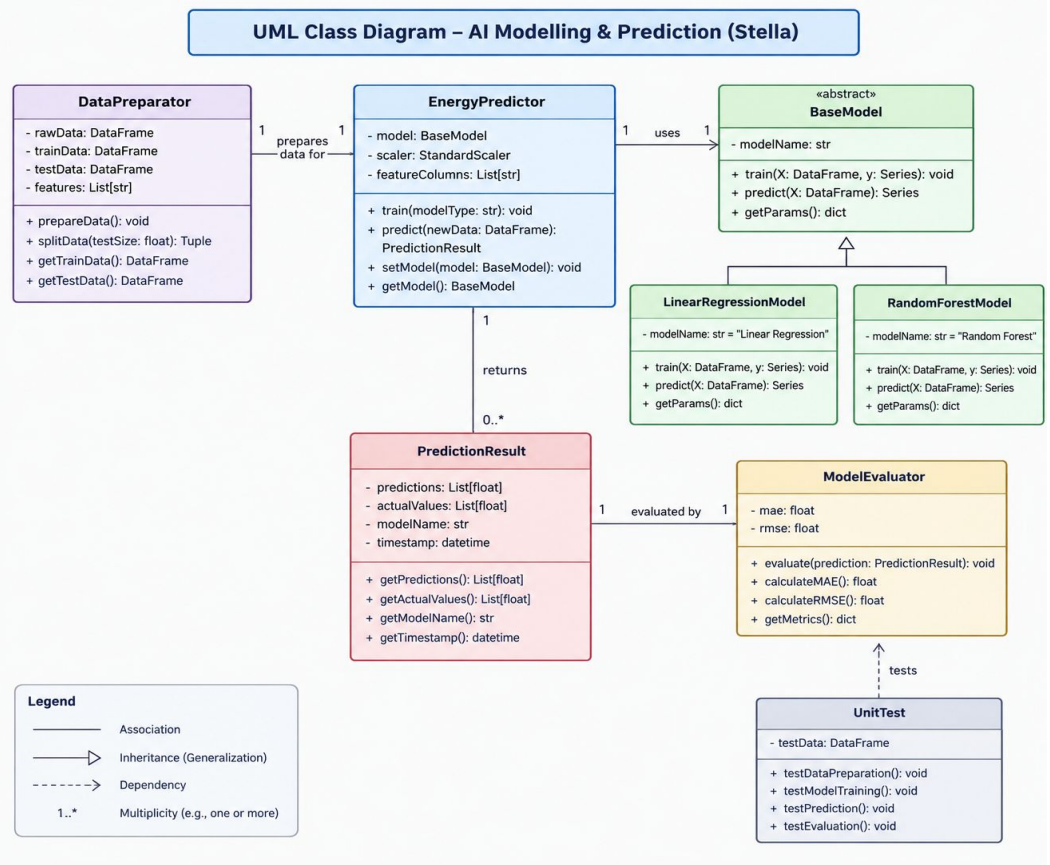
Subsystem 2: **AI modeling & Prediction**

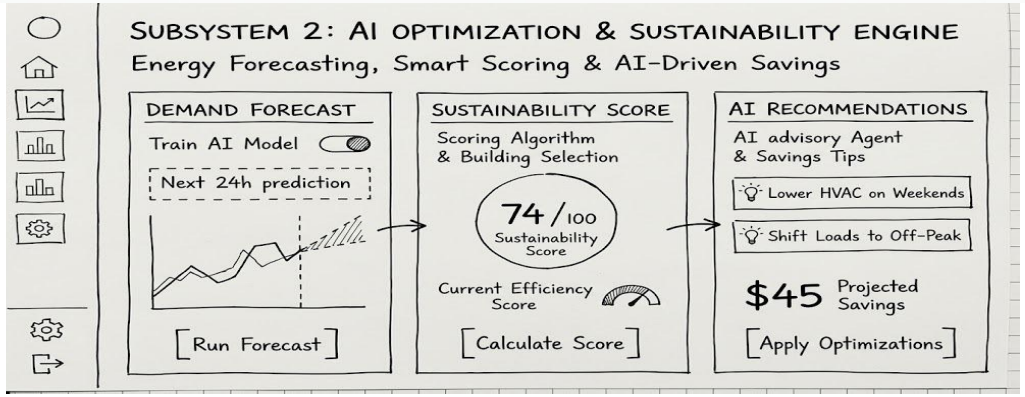
Functional Requirements

1. Prepares the dataset and divides it into training and testing data.
2. Trains Linear Regression and Random Forest models using training data.
3. Predicts energy consumption based on input data and trained models.
4. Evaluates prediction accuracy using MAE, RMSE.
5. Generates different scenarios based on input parameters and operating conditions.
6. Compares scenarios to identify differences in predicted energy consumption.
7. Identifies operating conditions that may reduce energy consumption.
8. Displays prediction results and evaluation metrics in a clear format.
9. Connects the prediction subsystem with other system components.
10. Tests interactions between integrated subsystems to identify errors.

Non-Functional Requirements

1. Tests interactions between integrated subsystems to identify errors.
2. Ensures predictions are sufficiently close to actual energy consumption values.
3. Performs model training and prediction within a reasonable time.
4. Allows the system to be updated and modified easily..
5. Ensures consistent operation and handles unexpected inputs appropriately.
6. Supports larger datasets and increasing workloads.
7. Presents results in a clear and understandable way for users.
8. Allows models and components to be reused for different prediction tasks.
9. Enables system functions and prediction performance to be tested effectively.
10. Ensures prediction outputs and evaluation results follow consistent formats and methods.





Subsystem 2: Stella (Sung Eun Kim)

Subsystem 3: **Optimisation, Visualisation & Integration**

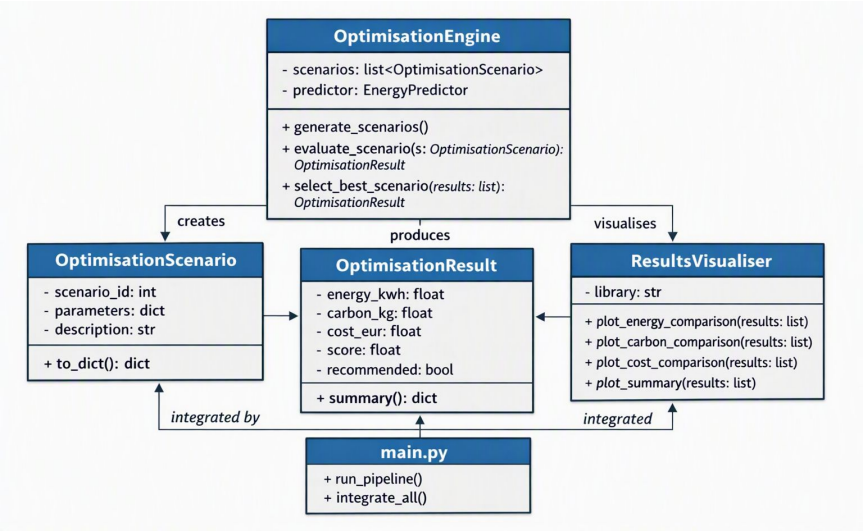
Functional Requirements

1. Generate multiple energy-saving scenarios.
2. Evaluate each scenario using AI predictions.
3. Compute energy consumption for each scenario.
4. Compute carbon emissions for each scenario.
5. Compute operating costs for each scenario.
6. Compare scenarios using quantitative metrics.
7. Recommend the best scenario based on optimisation criteria.
8. Visualise scenario comparisons using charts.
9. Produce structured optimisation results for integration.
10. Integrate all subsystem outputs into the main application.

Non-Functional Requirements

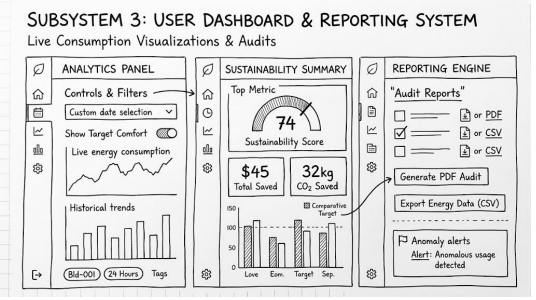
1. Generate optimisation results within 2 seconds.
2. Handle invalid or missing scenario inputs gracefully.
3. Maintain consistent data formats across subsystems.
4. Ensure visualisations render within 1 second.
5. Follow PEP8 coding standards.
6. Support modular extension of optimisation strategies.
7. Provide clear error messages for failed evaluations.
8. Ensure reproducible optimisation results.
9. Maintain compatibility with Matplotlib and Plotly.
10. Integrate seamlessly with main.py without breaking other modules.

Class Diagram



Interface Prototype

Visualisation or the interface shall be done by Matplotlib/Plotly charts



Subsystem 3: Devashish Deshpande